

▼ 快速入门：

- 镜像与容器：

▼ docker run 命令：

- 选项详解：

▼ Docker基础：

- 基础命令：

- 演示：

▼ 数据卷：

- 案例1-利用Nginx容器部署静态资源：
- 案例2-MySQL容器数据挂载：

▼ 自定义镜像：

- 简介：
- Dockerfile：
- 镜像构建：

- 网络：

▼ 项目部署：

- DockerCompose：

快速入门：

镜像与容器：

当利用Docker安装应用时，Docker会自动搜索并下载应用**镜像 (image)**。镜像不仅包含应用本身，还包含应用运行所需要的环境、配置、系统函数库。Docker会在运行镜像时创建一个隔离环境，称为**容器 (container)**。

镜像仓库：存储和管理镜像的平台，Docker官方维护的公共仓库：[Docker Hub](#)

docker run 命令：

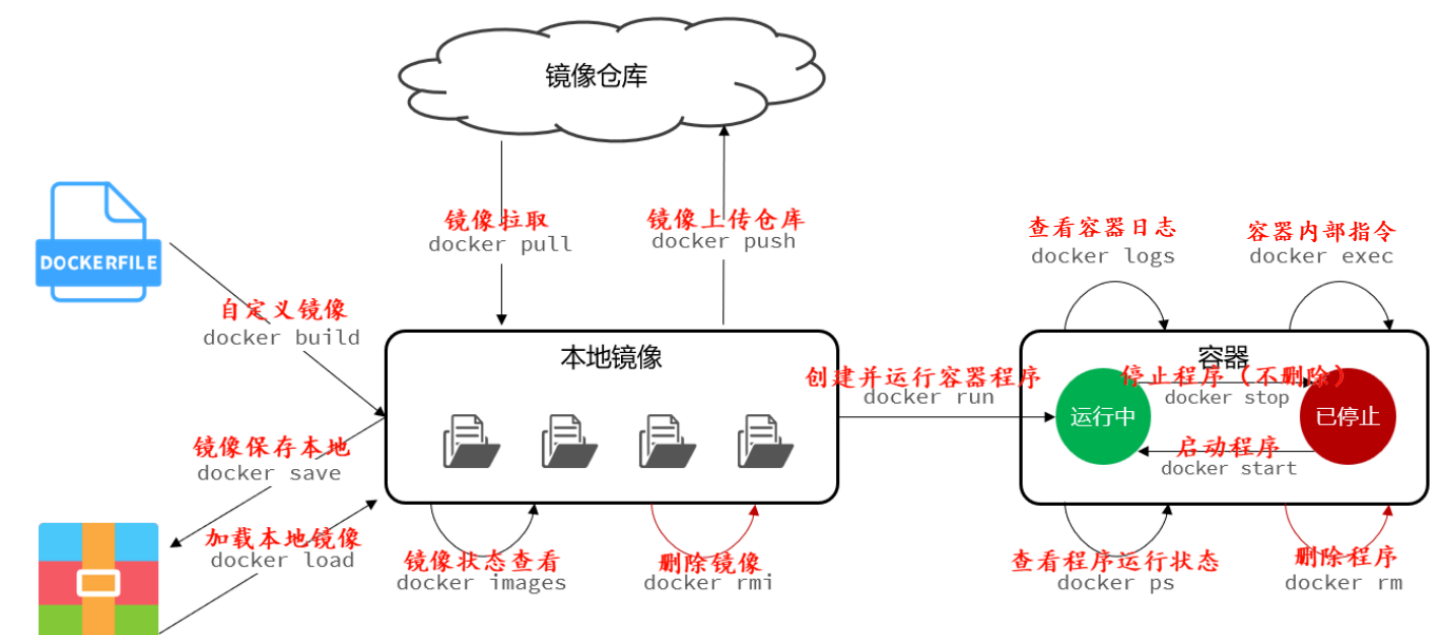
```
docker run -d \  
  --name mysql \  
  -p 3306:3306 \  
  -e TZ=Asia/Shanghai \  
  -e MYSQL_ROOT_PASSWORD=123 \  
  mysql
```

选项详解：

选项	描述	示例
-d	容器后台运行	docker run -d \
-- name 名称	为容器命名	--name nginx \
-e 环境变量	环境变量（查阅官方文档）	--
-p 宿主机端口：容器内端口	宿主机端口映射到容器内端口	-p 80： 80 \
Repository（镜像名）：TAG（版本号）	镜像名称结构	nginx

Docker基础：

基础命令：



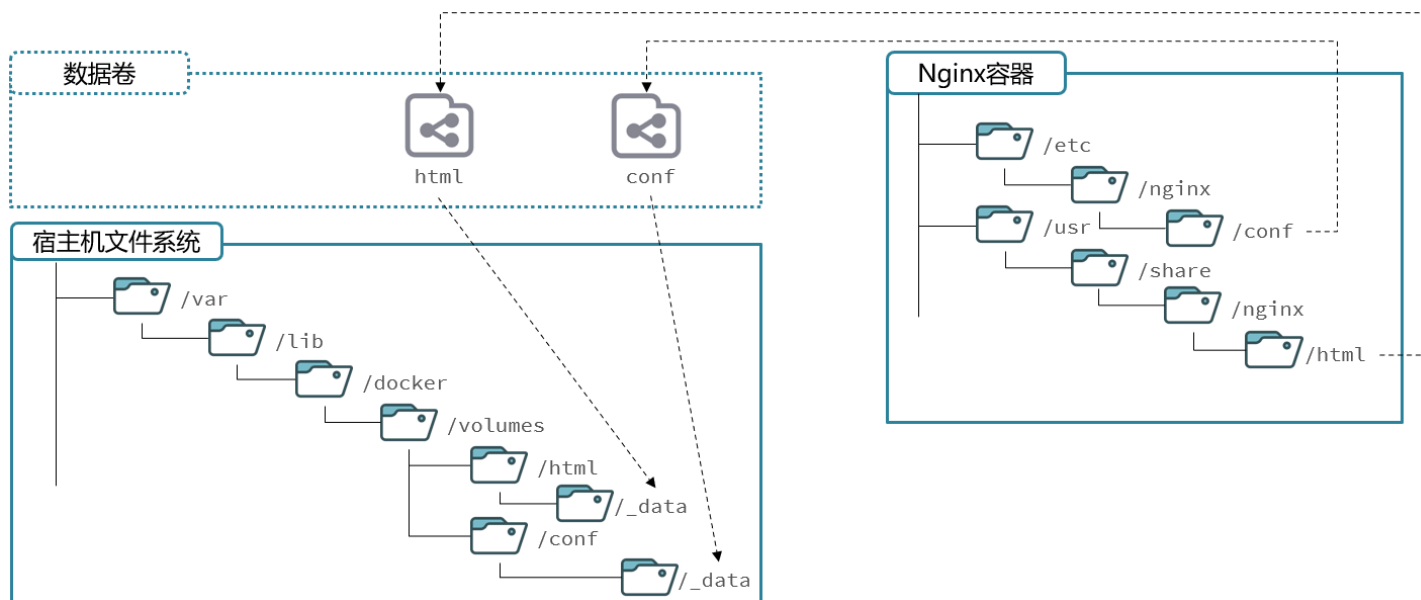
官方文档：[Docker 命令官方文档](#)

演示：

```
# 拉取Nginx镜像,创建并运行Nginx容器
1. 在DockerHub中搜索Nginx镜像,查看镜像的名称
https://hub.docker.com/_/nginx
2. 拉取Nginx镜像(注意配置镜像加速)
docker pull nginx
# 保存镜像到本地文件
docker save [-o 文件路径] 镜像名 [镜像:版本]
docker save -o nginx.tar nginx:latest
3. 查看本地镜像列表
docker images
4. 创建并运行Nginx容器
docker run -d --name nginx -p 80:80 nginx
5. 查看容器信息
docker ps [-a]
-a 查看所有容器信息(包括未启动容器)
# 查看指定容器日志
docker logs [-f] nginx
-f 持续跟踪查看
6. 停止容器
docker stop nginx
7. 重新启动容器
docker start nginx
8. 进入Nginx容器
docker exec -it nginx bash
9. 删除容器
docker rm 容器名 -f
-f 强制删除(否则需要先将容器停止运行再删除)
# 加载本地镜像
docker load [-i 镜像名]
docker load -i nginx.tar
```

数据卷：

数据卷 (volume) 是虚拟目录，是容器内目录与宿主机目录之间映射的桥梁，建立数据卷即可利用宿主机中的高级 (Linux) 命令同步修改容器目录中的对应文件以及部署静态资源



命令	功能	文档地址
docker volume create	创建数据卷	docker volume create
docker volume ls	查看所有数据卷	docker volume ls
docker volume rm	删除指定数据卷	docker volume rm
docker volume inspect	查看某个数据卷的详情	docker volume inspect
docker volume prune	清除数据卷	docker volume prune

案例1-利用Nginx容器部署静态资源：

创建Nginx容器，修改Nginx容器内的 html目录 下的 index.html文件，查看变化
将静态资源部署到Nginx的 html目录

挂载Nginx容器内的html目录下的index.html文件

注意挂载数据卷必须在容器创建时便挂载, 否则只能将原有容器删除再执行 docker run命令

语法: docker run -v 数据卷名称:容器内目录

docker run -d --name nginx -p 80:80 -v html:/usr/share/nginx/html nginx

```
[root@sc-docker nfs-web]# docker volume inspect nfs-web-6
[
  {
    "CreatedAt": "2022-08-26T18:24:33+08:00",
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/nfs-web-6/_data",
    "Name": "nfs-web-6",
    "Options": {
      "device": ":/web",
      "o": "addr=192.168.1.133,nolock,soft,rw,sync",
      "type": "nfs"
    },
    "Scope": "local"
  }
]
```

案例2-MySQL容器数据挂载：

- 查看MySQL容器自带数据卷挂载
 - 基于宿主机目录实现MySQL数据目录、配置文件、初始化脚本的挂载（查阅官方镜像文档）
- ①挂载 /root/mysql/data 到容器内的 /var/lib/mysql目录
 - ②挂载 /root/mysql/conf 到容器内的 /etc/mysql/conf.d目录（携带配置文件）
 - ③挂载 /root/mysql/init 到容器内的 /docker-entrypoint-initdb.d目录（携带SQL脚本）

将MySQL容器内指定目录挂载到本地指定目录

语法：docker run -v 本地目录:容器内目录

注意区分数据卷挂载与本地目录挂载

docker run -v mysql:/var/lib/mysql 将识别为数据卷(mysql),挂载路径默认:/var/lib/docker/vol

docker run -v ./mysql:/var/lib/mysql 将指定挂载路径为当前目录下的mysql目录

自定义镜像：

简介：

构建镜像的过程其实就是把应用程序、程序运行的系统函数库、运行配置等文件包打包的过程

部署Java应用的步骤：

- ① 准备Linux服务器
- ② 安装JRE并配置环境变量
- ③ 拷贝Jar包
- ④ 运行Jar包

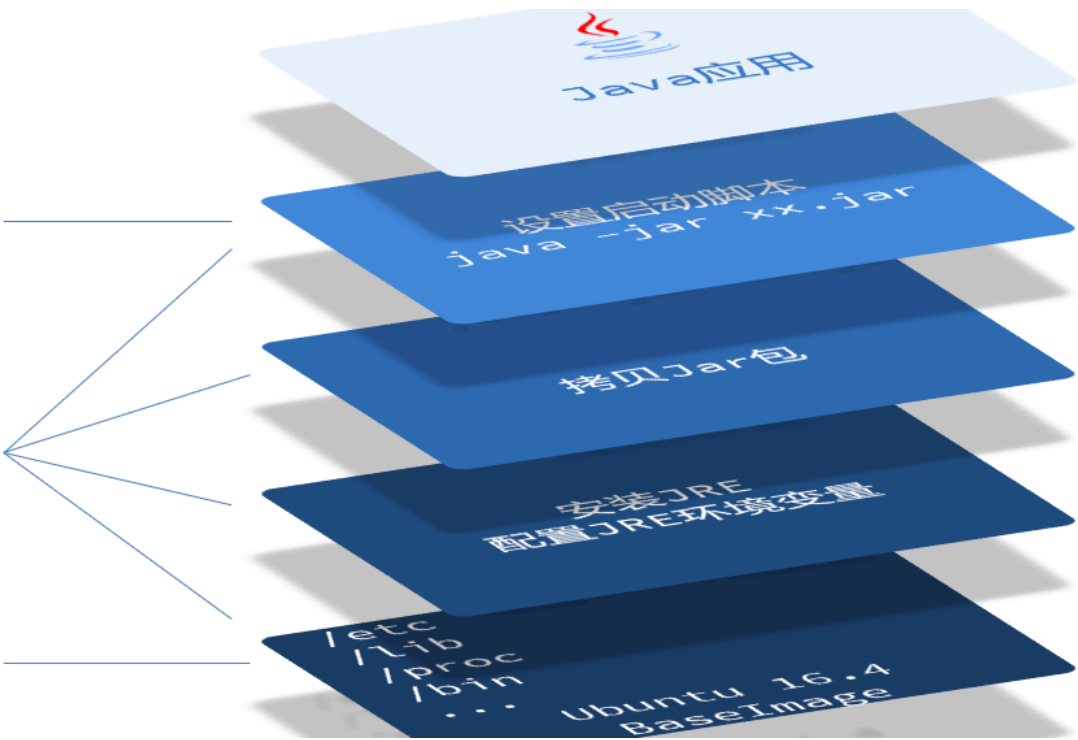
构建Java镜像的步骤：

- ① 准备Linux运行环境
- ② 安装JRE并配置环境变量
- ③ 拷贝Jar包
- ④ 编写运行脚本

将镜像分层打包便于组装与重构，提升代码的复用性，但是手动自定义镜像需要查询大量资料、打包各种压缩包，步骤较为繁琐，因此使用 Dockerfile 可以帮助构建镜像

镜像结构

- 入口 (EntryPoint)**
镜像运行入口，通常是程序启动的脚本和参数
- 层 (Layer)**
添加安装包、依赖、配置等，每次操作都形成新的层
- 基础镜像 (BaseImage)**
应用依赖的系统函数库、环境、配置、文件等



Dockerfile：

Dockerfile本质上为文本文件，其中包含众多**指令(Instruction)**，用指令来说明要执行的操作来构建镜像。将来Docker可以根据Dockerfile协助构建镜像

指令	功能	示例
FROM	指定基础镜像	FROM centos:6
ENV	设置环境变量	ENV key value
COPY	拷贝本地文件到镜像的指定目录	COPY ./jre11.tar.gz /tmp
RUN	执行Linux命令 (通常是安装过程的命令)	RUN tar -zxvf /tmp/jre11.tar.gz && EXPORTS path=/tmp/jre11:\$path
EXPOSE	指定容器运行时监听的端口	EXPOSE 8080
ENTRYPOINT	镜像应用的启动命令， 容器运行时调用	ENTRYPOINT java -jar xx.jar

详情参考官网文档：<https://docs.docker.com/engine/reference/builder>

```
# 基于Ubuntu基础镜像利用Dockerfile描述Java镜像结构
# 指定基础镜像
FROM ubuntu:16.04
# 配置环境变量,JDK的安装目录、容器内时区
ENV JAVA_DIR=/usr/local
# 拷贝JDK和Java项目的包
COPY ./jdk8.tar.gz $JAVA_DIR/
COPY ./docker-demo.jar /tmp/app.jar
# 安装JDK
RUN cd $JAVA_DIR \ && tar -xf ./jdk8.tar.gz \ && mv ./jdk1.8.0_144 ./java8
# 配置环境变量
ENV JAVA_HOME=$JAVA_DIR/java8
ENV PATH=$PATH:$JAVA_HOME/bin
# 入口,Java项目的启动命令
ENTRYPOINT ["java", "-jar", "/app.jar"]

-----

# 基于打包完毕的Java基础镜像构建自定义Java镜像
# 基础镜像
FROM openjdk:11.0-jre-buster
# 拷贝jar包
COPY docker-demo.jar /app.jar
# 入口
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

镜像构建：

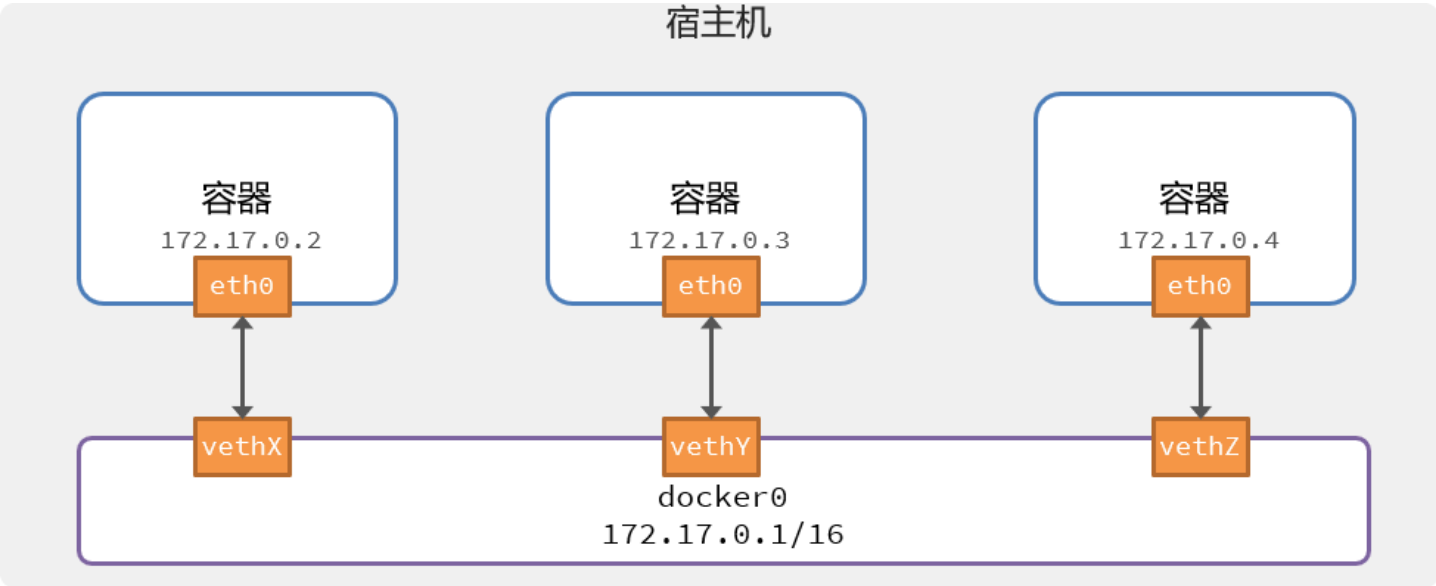
Dockerfile文件编写完成后，利用 build命令 构建镜像

```
# build —— 构建镜像
语法： docker build -t Repository:Tag Dockerfile所在目录
-t 即为镜像命名,格式:Repository:Tag,未指定Tag时默认为latest
Dockerfile文件通常与镜像位于同目录下,为此路径通常指定： .
```

```
# Java镜像构建演示
# 文件准备(jdk.tar、docker-demo.jar、Dockerfile)
# 拉取本地镜像
docker load -i jdk.tar
# 构建Java容器
cd demo/
docker build -t docker_java .
# 运行Java容器
docker run -d --name java_11.0 -p 8080:8080 docker_java
# 查看容器运行状态及运行日志
docker ps
docker logs -f java_11.0
```

网络：

默认情况下，所有容器都是以Bridge方式的连接到Docker的虚拟网桥上，但是Docker的虚拟地址可能会随宿主机的关闭启动而修改，为此可以使用自定义网络来更加方便地访问服务器：



加入自定义网络的容器还可以通过容器名互相访问，Docker的网络操作命令如下：

指令	功能	文档地址
docker network create 网络名	创建网络	docker network create
docker network ls	查看所有网络	docker network ls
docker network rm	删除指定网络	docker network rm
docker network prune	清除未使用的网络	docker network prune
docker network connect 网络名 容器名	使指定容器连接加入某网络	docker network connect

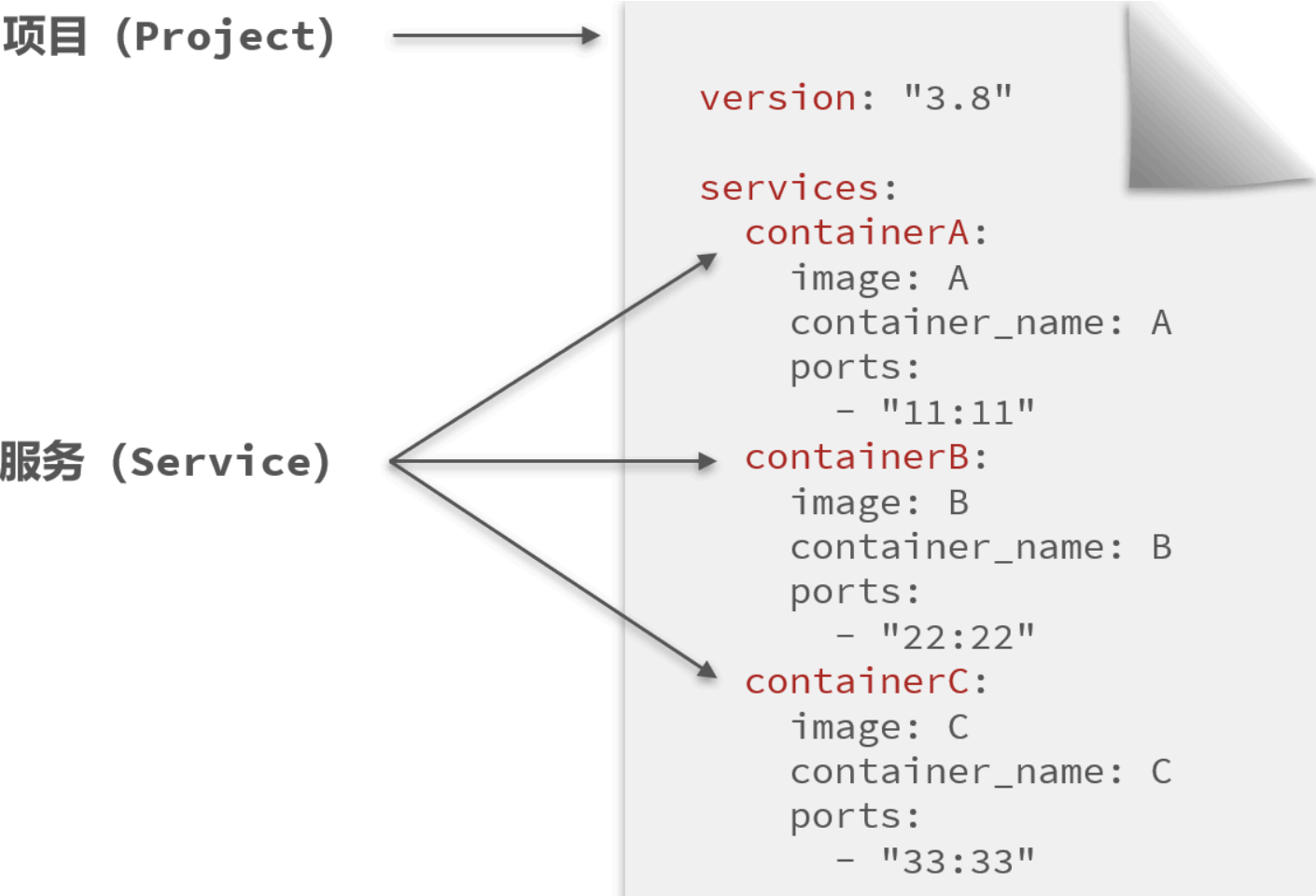
指令	功能	文档地址
docker network disconnect	使指定容器连接离开某网络	docker network disconnect
docker network inspect	查看网络详细信息	docker network inspect

```
# 创建自定义网络
docker network create tassel
# 使指定容器连接加入自定义网络
docker network connect tassel java_11.0
# 或在容器创建时即加入网络
docker run -d --name java_11.0 -p 8080:8080 --network tassel docker_java
# 查看网络连接情况
docker inspect java_11.0
# 进入docker_java容器检测同网桥互连
docker exec -it bash
ping 容器（容器需先加入tassel网桥）
```

项目部署：

DockerCompose：

Docker Compose通过单独的 **docker-compose.yml** 模板文件（YAML 格式）来定义相关联的应用容器，帮助我们实现**相互关联的Docker容器的快速部署**



命令格式

docker compose [OPTIONS] [COMMAND]

类型	参数或指令	功能
Options	-f	指定compose文件的路径与名称
Options	-p	指定project名称
Commands	up	创建并启动所有service容器
Commands	down	停止并移除所有容器、网络
Commands	ps	列出所有启动的容器
Commands	logs	查看指定容器的日志
Commands	stop	停止容器
Commands	start	启动容器
Commands	restart	重启容器
Commands	top	查看运行进程
Commands	exec	指定运行容器中执行命令

